

Implementation of a 512 bits randomized primality test

Andrea Bellani (andrea1.bellani@mail.polimi.it)

October 21, 2025

High-level sketch of primality tests

Naive approach

A naive approach to primality testing consists simply in firstly check whether a number is 0, 1 or even, then checking if any odd number from 3 to $\sqrt{n}$ divides it. Therefore, a prime number as input is the worst-case time complexity.

Randomized approach

Miller-Rabin primality test. If n is prime, then $\forall a, 0 < a < n, a^2 \bmod n = 1 \iff a = 1 \vee a = n - 1$

For a single a , this criterion can be verified in constant time (if we neglect bit-level operations cost), however we may verify it for $\Theta(n)$ as , which may result in a worse complexity than the naive approach. Our randomized test is going to verify the Miller-Rabin test for $O(\log n)$ and as as , we are going to use the intermediate results ($\bmod n$) obtained from the recursive calculus of a^n , where a is a randomly chosen number between 2 and $n - 1$.

Technical setup

Testing and complexity measurements are performed with Google Benchmark tools provided by the Colab notebook shared by Professor Ferrandi.

The random number generator is the 64 bits Mersenne Twister random number generator provided by Boost Random, that we use to randomly extract a number from the `uniform_int_distribution<uint512>` provided by Boost Random.

The only compiler flag used that may introduce optimizations is `-O2`, which acts at a more basic and general level. Finally, for complexity analysis, `return values` is passed to `DoNotOptimize` in order to prevent possible compiler optimizations on tests results (e.g. re-use for the following iterations).

Design choices

The implementation follows the pseudo-code presented in lessons, which we only trivially extended

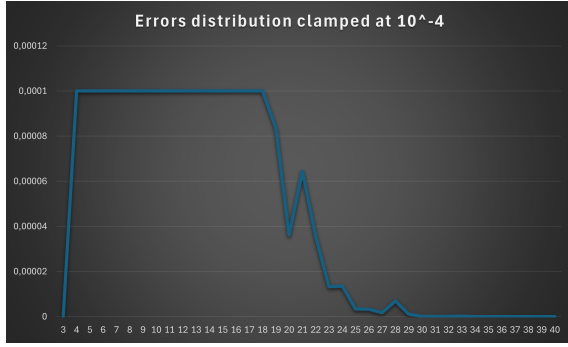
for $n < 2$. It is almost a straight-forward implementation from the pseudo-code shared by Professor Ferrandi, unless an unitary right bit shift for implementing $p/2$.

Tests and complexity analysis are performed as function of the number of bits used to represent the number tested or, if differently specified, of the number of limbs (which are 64 bit groups for our implementation). This choice has both the advantage of overcoming Google Benchmark limitation in passing `uint512` inputs as states and permits us to draw more readable "uniform" plots, since we do not distinguish between numbers represented with the same number of bits.

Correctness testing

From the theory, the only case that is worth to test the randomized tester, since it is Monte Carlo false-biased, is when a number is detected as composite by the test but the Naive (which we previously tested and from now on we are going to consider as "correct") detects the number as prime. We test the absence of this kind of errors for 1000 composite numbers represented each $n \in [3; 70]$ (the case $n \leq 2$ is only characterized by trivial composites or prime numbers). Similarly we could have test if numbers detected as prime by the Naive tester are detected as prime also by the randomized tester.

Another interesting result may be to verify how many errors are performed in the recognition of composite numbers (prime numbers are always detected as prime by the randomized test). So, What we do is to calculate multiple times how many recognition errors change as numbers size in bits changes. In order to provide a statistical sounding result we checked for every number of bits from 3 to 40, 10^4 composite numbers, for 10^3 executions each (greater numbers of executions or of extracted numbers revealed negligible differences in results). If we perform the average probability that an execution is wrong with the numbers extracted and consider reliable our test if the probability of making a mistake is $\leq 10^{-4}$, as far as we measured, we can consider our randomized test reliable only for ≥ 19 bits numbers. Here we show the results obtained (probability values above

Figure 1: Errors distribution clamped at 10^{-4}

10^{-4} are clamped at 10^{-4} since they are out-of-scale and would penalize plots readability. More tolerant thresholds may consider reliable the test even for < 19 bits numbers.

Performance analysis

From a theoretical point of view, the randomized test would have a complexity of $O(n^3)$ (where n is the number of bits used to represent the input) if we consider the schoolbook multiplication ($O(n^2)$).

Due to the possible optimizations and the greater variability introduced by the random choice of a , a possible way of providing a more reliable estimation of the complexity is to split the calculus up into two:

- the multiplication cost as bits number grows;
- the recursive cost (where any multiplication or division is substituted with a constant time expression) as bits number grows.

If we compare the effective complexity of the full algorithm and the product between the two split complexities properly tuned with a multiplicative factor, the split up seem to have effectively decoupled costs dependent by the number of bits (the curve approximated by the product follows the one of the measured complexity):

	Benchmark detected complexity	Benchmark RMS
Recursion Cost	$28.07 N$	17%
Multiplication with module Cost	$55.41 \log(N)$	45%
Full algorithm	$863.22 N$	36%

Table 1: Asymptotic complexities detected by the benchmark with RMSs

As we can see from the plots, recursion as the expected complexity of $O(n)$, while multiplication is piecewise constant, where plateaus seem to grow linearly with n . Therefore the total complexity falls in

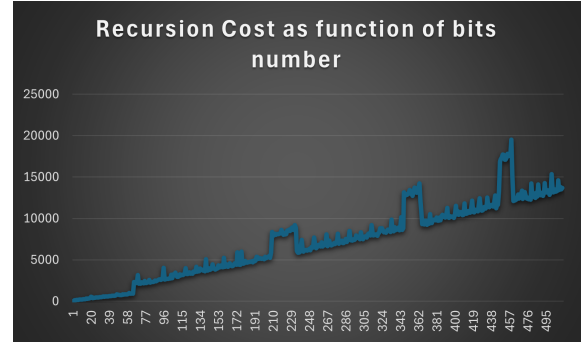


Figure 2: Recursion cost as function of bits number

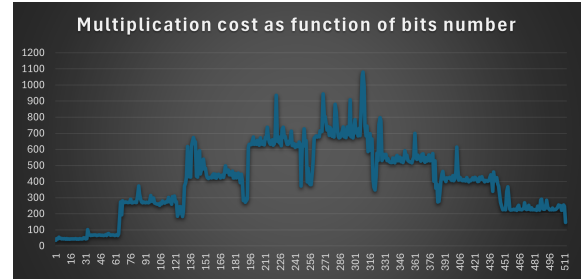


Figure 3: Multiplication cost as function of bits number

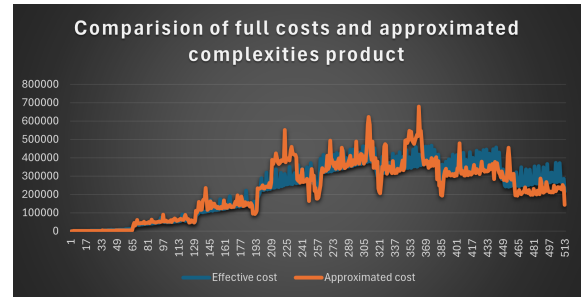


Figure 4: Full cost comparison between recursion and multiplication product and effective measurement

$O(n^2)$ class however if we plot multiplication complexity as function of limbs number we see that in the range of 512 bits (i.e. 8 limbs) the complexity is $O(n^2)$, where n is the number of limbs. However, it seems nearly constant or linear if we analyze it in the very first part.

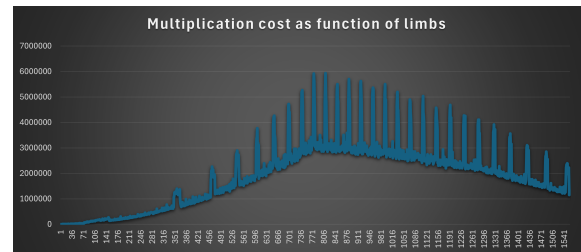


Figure 5: Multiplication cost as function of limbs number